

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\differentiation\FiniteDifferencesDifferentiator.java

- Specification

module FiniteDifferencesDifferentiator

type

```
nPts: int           // number of stencil points
h: real             // step size (strictly positive)
a, b: real          // lower/upper bounds, a < b
span: real          // (nPts - 1) * h
x: real             // evaluation abscissa for double path
y: real             // scalar result
ex: set of {
  None,
  NullArgumentException,
  NotStrictlyPositiveException,
  NumberIsTooSmallException,
  NumberIsTooLargeException,
  OutOfRangeException,
  NotFiniteNumberException
}
```

process Construct(nPts: int, h: real, a: real, b: real) result: (span: real, ex: set)

Functional Scenario Form

```
nPts < 2                                && ex = NumberIsTooSmallException
||
h <= 0                                  && ex = NotStrictlyPositiveException
||
a >= b                                  && ex =
NumberIsTooLargeException ||
nPts * h > b - a                        && ex = NumberIsTooLargeException ||
nPts > 100                              && (ex = None && span = (nPts - 1) * h)
```

end_process

process Differentiate(f: UnivariateFunction) result: (g: UnivariateDifferentiableFunction, ex: set)

Functional Scenario Form

```
(f = null)                                && (ex = NullArgumentException) or
(f ≠ null)                               && (ex = None && g wraps(f) with
finite-difference stencil of length nPts and step h, domain [a,b])
```

end_process

process Eval0(g, x: real) result: (y: real, ex: set)

Functional Scenario Form

(x is NaN) && (ex = None && isNaN(y)) or
(x = $\pm \infty$) && (ex =
NotFiniteNumberException) or
(x < a || x > b) && (ex = OutOfRangeException) or
(a $\leq$ x $\leq$ b && no NaN produced by f at sampled stencil points)
&& (ex = None && y = finite-
difference combination of f(x + k*h_shifted))
// h_shifted: stencil may be
shifted so that all sample points $\in$ [a,b]
(a $\leq$ x $\leq$ b && any sampled f value is NaN) && (ex = None && isNaN(y)) or
(a $\leq$ x $\leq$ b && arithmetic over/underflow) && (ex = None && y may be $\pm$
Infinity or signed zero per IEEE-754)

end_process

process EvalDS(g, t: DerivativeStructure) result: (T: DerivativeStructure, ex: set)

Functional Scenario Form

(t = null) && (ex = NullArgumentException) or
(t.value is NaN) && (ex = None && T is NaN DS) or
(t.value = $\pm \infty$) && (ex =
NotFiniteNumberException) or
(t.value < a || t.value > b) && (ex = OutOfRangeException) or
(t.order > (nPts - 1)) && (ex = NumberIsTooLargeException)
or
(a $\leq$ t.value $\leq$ b && t.order $\leq$ (nPts - 1) && all sampled values finite)
&& (ex = None && T = DS
computed by finite-difference weights up to order t.order) or
(a $\leq$ t.value $\leq$ b && any sampled f is NaN) && (ex = None && T is NaN DS)

end_process

process StencilShift(x: real) result: (center: real, indices: sequence of int)

Functional Scenario Form

(a + span/2 $\leq$ x $\leq$ b - span/2) && (center = x; indices =
symmetric offsets around center) or
(x < a + span/2) && (center = a + span/2; indices
shifted right so that min sample = a, max sample = a + span) or
(x > b - span/2) && (center = b - span/2; indices
shifted left so that min sample = b - span, max sample = b)
end_process

process MaxOrder(nPts: int) result: (maxOrd: int)

Functional Scenario Form

(nPts $\geq$ 2)

&& (maxOrd = nPts - 1)

end_process

end_module

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 * http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.differentiation;
import org.apache.commons.math4.legacy.analysis.UnivariateFunction;
import org.apache.commons.math4.legacy.analysis.UnivariateMatrixFunction;
import org.apache.commons.math4.legacy.analysis.UnivariateVectorFunction;
import org.apache.commons.math4.legacy.exception.MathIllegalArgumentExc
eption;
import org.apache.commons.math4.legacy.exception.NotPositiveException;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;
import org.apache.commons.math4.legacy.exception.NumberIsTooSmallExcept
ion;
import org.apache.commons.math4.core.jdkmath.JdkMath;
import org.apache.commons.math4.legacy.analysis.UnivariateFunction;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
```

```

import org.apache.commons.math4.legacy.exception.NotStrictlyPositiveException;
import org.apache.commons.math4.legacy.exception.NumberIsTooSmallException;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeException;
import org.apache.commons.math4.legacy.exception.OutOfRangeException;
import org.apache.commons.math4.legacy.exception.NotFiniteNumberException;
import java.util.Locale;
import java.util.Arrays;
/**
 * Univariate functions differentiator using finite differences.
 * <p>
 * This class creates some wrapper objects around regular
 * {@link UnivariateFunction univariate functions} (or
 * {@link UnivariateVectorFunction univariate vector functions} or
 * {@link UnivariateMatrixFunction univariate matrix functions}). These
 * wrapper
 * objects compute derivatives in addition to function values.
 * </p>
 * <p>
 * The wrapper objects work by calling the underlying function on a sampling
 * grid around the current point and performing polynomial interpolation. A
 * finite differences scheme with n points is theoretically able to compute
 * derivatives up to order n-
1, but it is generally better to have a slight
 * margin. The step size must also be small enough in order for the polynomial
 * approximation to be good in the current point neighborhood, but it should not
 * be too small because numerical instability appears quickly (there are several
 * differences of close points). Choosing the number of points and the step size
 * is highly problem dependent.
 * </p>
 * <p>
 * As an example of good and bad settings, let's consider the quintic polynomial
 * function {@code f(x) = (x-1)*(x-0.5)*x*(x+0.5)*(x+1)}. Since it is a

```

```

* polynomial, finite differences with at least 6 points should theoret
ically
* recover the exact same polynomial and hence compute accurate derivat
ives for
* any order. However, due to numerical errors, we get the following re
sults for
* a 7 points finite differences for abscissae in the [-10, 10] range:
* <ul>
* <li>step size = 0.25, second order derivative error about 9.97e-
10</li>
* <li>step size = 0.25, fourth order derivative error about 5.43e-
8</li>
* <li>step size = 1.0e-6, second order derivative error about 148</li>
* <li>step size = 1.0e-
6, fourth order derivative error about 6.35e+14</li>
* </ul>
* <p>
* This example shows that the small step size is really bad, even simp
ly for
* second order derivative!
* </p>
*
* @since 3.1
*/
public class FiniteDifferencesDifferentiator implements UnivariateFunct
ionDifferentiator,
    UnivariateVectorFunctionDifferentiator, UnivariateMatrixFunctionDiffe
rentiator {
    /** Number of points to use. */
    private final int nbPoints;
    /** Step size. */
    private final double stepSize;
    /** Half sample span. */
    private final double halfSampleSpan;
    /** Lower bound for independent variable. */
    private final double tMin;
    /** Upper bound for independent variable. */
    private final double tMax;
    /**
     * Build a differentiator with number of points and step size when ind
ependent
     * variable is unbounded.
     * <p>

```

```

    * Beware that wrong settings for the finite differences differentiator can lead
    * to highly unstable and inaccurate results, especially for high derivation
    * orders. Using very small step sizes is often a bad idea.
    * </p>
    *
    * @param nbPoints number of points to use
    * @param stepSize step size (gap between each point)
    * @exception NotPositiveException if {@code stepsize <= 0} (note
that
    *                                     {@link NotPositiveException} extends
ds
    *                                     {@link NumberIsTooSmallException})
    * @exception NumberIsTooSmallException {@code nbPoint <= 1}
    */
    public FiniteDifferencesDifferentiator(final int nbPoints, final double stepSize)
        throws NotPositiveException, NumberIsTooSmallException {
        this(nbPoints, stepSize, Double.NEGATIVE_INFINITY, Double.POSITIVE_INFINITY);
    }
    /**
    * Build a differentiator with number of points and step size when independent
    * variable is bounded.
    * <p>
    * When the independent variable is bounded ( $t_{\text{Lower}} < t < t_{\text{Upper}}$ ), the
    * sampling points used for differentiation will be adapted to ensure the
    * constraint holds even near the boundaries. This means the sample will not be
    * centered anymore in these cases. At an extreme case, computing derivatives
    * exactly at the lower bound will lead the sample to be entirely on the right
    * side of the derivation point.
    * </p>
    * <p>
    * Note that the boundaries are considered to be excluded for function
    * evaluation.
    * </p>
    * <p>

```

```

    * Beware that wrong settings for the finite differences differentiator
    can lead
    * to highly unstable and inaccurate results, especially for high derivation
    * orders. Using very small step sizes is often a bad idea.
    * </p>
    *
    * @param nbPoints number of points to use
    * @param stepSize step size (gap between each point)
    * @param tLower lower bound for independent variable (may be
    *               {@code Double.NEGATIVE_INFINITY} if there are no lower
    *               bounds)
    * @param tUpper upper bound for independent variable (may be
    *               {@code Double.POSITIVE_INFINITY} if there are no upper
    *               bounds)
    * @exception NotPositiveException if {@code stepsize <= 0} (note
    that
    *               {@link NotPositiveException} extends
    *               {@link NumberIsTooSmallException})
    * @exception NumberIsTooSmallException {@code nbPoints <= 1}
    * @exception NumberIsTooLargeException {@code stepSize * (nbPoints -
    1) >= tUpper - tLower}
    */
    public FiniteDifferencesDifferentiator(final int nbPoints, final double
    stepSize, final double tLower,
        final double tUpper) throws NotPositiveException, NumberIsTooSmallException,
    NumberIsTooLargeException {
        if (nbPoints <= 1) {
            throw new NumberIsTooSmallException(stepSize, 1, false);
        }
        this.nbPoints = nbPoints;
        if (stepSize <= 0) {
            throw new NotPositiveException(stepSize);
        }
        this.stepSize = stepSize;
        halfSampleSpan = 0.5 * stepSize * (nbPoints - 1);
        if (2 * halfSampleSpan >= tUpper - tLower) {
            throw new NumberIsTooLargeException(2 * halfSampleSpan, tUpper - tLower,
        false);
        }
        final double safety = JdkMath.ulp(halfSampleSpan);

```

```

    this.tMin = tLower + halfSampleSpan + safety;
    this.tMax = tUpper - halfSampleSpan - safety;
}
/**
 * Get the number of points to use.
 *
 * @return number of points to use
 */
public int getNbPoints() {
    return nbPoints;
}
/**
 * Get the step size.
 *
 * @return step size
 */
public double getStepSize() {
    return stepSize;
}
/**
 * Evaluate derivatives from a sample.
 * <p>
 * Evaluation is done using divided differences.
 * </p>
 *
 * @param t evaluation abscissa value and derivatives
 * @param t0 first sample point abscissa
 * @param y function values sample
 *          {@code y[i] = f(t[i]) = f(t0 + i * stepSize)}
 * @return value and derivatives at {@code t}
 * @exception NumberIsTooLargeException if the requested derivation order is
 *                                     larger or equal to the number of points
 */
private DerivativeStructure evaluate(final DerivativeStructure t, final double t0, final double[] y)
    throws NumberIsTooLargeException {
    // create divided differences diagonal arrays
    final double[] top = new double[nbPoints];
    final double[] bottom = new double[nbPoints];
    for (int i = 0; i < nbPoints; ++i) {
        // update the bottom diagonal of the divided differences array
        bottom[i] = y[i];
    }
}

```

```

        for (int j = 1; j <= i; ++j) {
            bottom[i - j] = (bottom[i - j + 1] - bottom[i - j]) / (j * stepSize);
        }
        // update the top diagonal of the divided differences array
        top[i] = bottom[0];
    }
    // evaluate interpolation polynomial (represented by top diagonal) at
    t
    final int order = t.getOrder();
    final int parameters = t.getFreeParameters();
    final double[] derivatives = t.getAllDerivatives();
    final double dt0 = t.getValue() - t0;
    DerivativeStructure interpolation = new DerivativeStructure(parameters, order, 0.0);
    DerivativeStructure monomial = null;
    for (int i = 0; i < nbPoints; ++i) {
        if (i == 0) {
            // start with monomial(t) = 1
            monomial = new DerivativeStructure(parameters, order, 1.0);
        } else {
            // monomial(t) = (t - t0) * (t - t1) * ... * (t - t(i-1))
            derivatives[0] = dt0 - (i - 1) * stepSize;
            final DerivativeStructure deltaX = new DerivativeStructure(parameters, order, derivatives);
            monomial = monomial.multiply(deltaX);
        }
        interpolation = interpolation.add(monomial.multiply(top[i]));
    }
    return interpolation;
}
/**
 * {@inheritDoc}
 * <p>
 * The returned object cannot compute derivatives to arbitrary orders.
 * The value
 * function will throw a {@link NumberIsTooLargeException} if the requested
 * derivation order is larger or equal to the number of points.
 * </p>
 */
@Override
public UnivariateDifferentiableFunction differentiate(final UnivariateFunction function) {

```

```

return new UnivariateDifferentiableFunction() {
    /** {@inheritDoc} */
    @Override
    public double value(final double x) throws MathIllegalArgumentException
ion {
        return function.value(x);
    }
    /** {@inheritDoc} */
    @Override
    public DerivativeStructure value(final DerivativeStructure t) throws
MathIllegalArgumentException {
        // check we can achieve the requested derivation order with the sam
ple
        if (t.getOrder() >= nbPoints) {
            throw new NumberIsTooLargeException(t.getOrder(), nbPoints, false)
;
        }
        // compute sample position, trying to be centered if possible
        final double t0 = JdkMath.max(JdkMath.min(t.getValue(), tMax), tMin
) - halfSampleSpan;
        // compute sample points
        final double[] y = new double[nbPoints];
        for (int i = 0; i < nbPoints; ++i) {
            y[i] = function.value(t0 + i * stepSize);
        }
        // evaluate derivatives
        return evaluate(t, t0, y);
    }
};
}
/**
 * {@inheritDoc}
 * <p>
 * The returned object cannot compute derivatives to arbitrary orders.
The value
 * function will throw a {@link NumberIsTooLargeException} if the requ
ested
 * derivation order is larger or equal to the number of points.
 * </p>
 */
@Override
public UnivariateDifferentiableVectorFunction differentiate(final Univ
ariateVectorFunction function) {
    return new UnivariateDifferentiableVectorFunction() {

```

```

    /** {@inheritDoc} */
    @Override
    public double[] value(final double x) throws MathIllegalArgumentException
    {
        return function.value(x);
    }
    /** {@inheritDoc} */
    @Override
    public DerivativeStructure[] value(final DerivativeStructure t) throws MathIllegalArgumentException {
        // check we can achieve the requested derivation order with the sample
        if (t.getOrder() >= nbPoints) {
            throw new NumberIsTooLargeException(t.getOrder(), nbPoints, false);
        }
        // compute sample position, trying to be centered if possible
        final double t0 = JdkMath.max(JdkMath.min(t.getValue(), tMax), tMin) - halfSampleSpan;
        // compute sample points
        double[][] y = null;
        for (int i = 0; i < nbPoints; ++i) {
            final double[] v = function.value(t0 + i * stepSize);
            if (i == 0) {
                y = new double[v.length][nbPoints];
            }
            for (int j = 0; j < v.length; ++j) {
                y[j][i] = v[j];
            }
        }
        // evaluate derivatives
        final DerivativeStructure[] value = new DerivativeStructure[y.length];
        for (int j = 0; j < value.length; ++j) {
            value[j] = evaluate(t, t0, y[j]);
        }
        return value;
    }
}
/**
 * {@inheritDoc}
 * <p>

```

```

    * The returned object cannot compute derivatives to arbitrary orders.
    The value
    * function will throw a {@link NumberIsTooLargeException} if the requested
    * derivation order is larger or equal to the number of points.
    * </p>
    */
    @Override
    public UnivariateDifferentiableMatrixFunction differentiate(final UnivariateMatrixFunction function) {
        return new UnivariateDifferentiableMatrixFunction() {
            /** {@inheritDoc} */
            @Override
            public double[][] value(final double x) throws MathIllegalArgumentException {
                return function.value(x);
            }
            /** {@inheritDoc} */
            @Override
            public DerivativeStructure[][] value(final DerivativeStructure t) throws MathIllegalArgumentException {
                // check we can achieve the requested derivation order with the sample
                if (t.getOrder() >= nbPoints) {
                    throw new NumberIsTooLargeException(t.getOrder(), nbPoints, false);
                }
                // compute sample position, trying to be centered if possible
                final double t0 = JdkMath.max(JdkMath.min(t.getValue(), tMax), tMin) - halfSampleSpan;
                // compute sample points
                double[][][] y = null;
                for (int i = 0; i < nbPoints; ++i) {
                    final double[][] v = function.value(t0 + i * stepSize);
                    if (i == 0) {
                        y = new double[v.length][v[0].length][nbPoints];
                    }
                    for (int j = 0; j < v.length; ++j) {
                        for (int k = 0; k < v[j].length; ++k) {
                            y[j][k][i] = v[j][k];
                        }
                    }
                }
                // evaluate derivatives

```

```

        final DerivativeStructure[][] value = new DerivativeStructure[y.length][y[0].length];
        for (int j = 0; j < value.length; ++j) {
            for (int k = 0; k < y[j].length; ++k) {
                value[j][k] = evaluate(t, t0, y[j][k]);
            }
        }
        return value;
    }
};
}
}
class FDD_VT {
    // ----- helpers -----
    private static void headline(String name) {
        System.out.println("\n==== " + name + " ====");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    private static void checkEquals(String name, double expected, double actual, double tolRel) {
        boolean ok;
        if (Double.isNaN(expected) || Double.isNaN(actual))
            ok = (Double.isNaN(expected) && Double.isNaN(actual));
        else if (Double.isInfinite(expected) || Double.isInfinite(actual))
            ok = (Double.doubleToRawLongBits(expected) == Double.doubleToRawLongBits(actual));
        else {
            double tol = Math.max(1.0, Math.max(Math.abs(expected), Math.abs(actual))) * tolRel;
            ok = (expected == actual) || (Math.abs(expected - actual) <= tol);
        }
    }
}

```

```

    System.out.printf("[%s] %s expected=%s actual=%s%n", ok ? "PASS" : "
FAIL", name, fmt(expected), fmt(actual));
}
private static void checkInt(String name, int exp, int act) {
    System.out.printf("[%s] %s expected=%d actual=%d%n", (exp == act) ?
"PASS" : "FAIL", name, exp, act);
}
private static String fmt(double v) {
    if (Double.isNaN(v))
        return "NaN";
    if (v == Double.POSITIVE_INFINITY)
        return "+Inf";
    if (v == Double.NEGATIVE_INFINITY)
        return "-Inf";
    if (v == 0.0)
        return (Double.doubleToRawLongBits(v) >>> 63) == 1 ? "-0.0" : "0.0";
    return Double.toString(v);
}
// ----- sample functions -----
private static final UnivariateFunction F_CONST5 = x -> 5.0;
private static final UnivariateFunction F_LIN = x -> 2.0 * x - 3.0; //
f' = 2
private static final UnivariateFunction F_QUAD = x -> x * x; // f' = 2
x, f''=2
private static final UnivariateFunction F_CUBIC = x -> x * x * x; // f
' = 3x^2, f''=6x, f'''=6
private static final UnivariateFunction F_SIN = Math::sin;
private static final UnivariateFunction F_NAN = x -> Double.NaN;
// ----- key fix: disambiguate overload -----
private static UnivariateDifferentiableFunction diff(FiniteDifferences
Differentiator D, UnivariateFunction f) {
    // Explicitly select the overload that takes legacy.analysis.Univaria
teFunction
    return D.differentiate((org.apache.commons.math4.legacy.analysis.Univ
ariateFunction) f);
}
public void Vtest() {
    // VT01: ctor nPts<2 -> NumberIsTooSmallException
    expectThrow("VT01 ctor(nPts<2)", () -> new FiniteDifferencesDifferent
iator(1, 0.1, 0.0, 1.0),
        NumberIsTooSmallException.class);
    // VT02: ctor h<=0 -> NotStrictlyPositiveException
    expectThrow("VT02 ctor(h<=0)", () -> new FiniteDifferencesDifferentia
tor(5, 0.0, 0.0, 1.0),

```

```

        NotStrictlyPositiveException.class);
    // VT03: ctor a>=b -> NumberIsTooLargeException
    expectThrow("VT03 ctor(a>=b)", () -> new FiniteDifferencesDifferentia
tor(5, 0.1, 1.0, 1.0),
        NumberIsTooLargeException.class);
    // VT04: ctor span>(b-a) -> NumberIsTooLargeException
    expectThrow("VT04 ctor(span > width)", () -> new FiniteDifferencesDif
ferentiator(5, 0.4, 0.0, 1.0),
        NumberIsTooLargeException.class);
    // VT05: ctor ok
    headline("VT05 ctor ok + basic");
    FiniteDifferencesDifferentiator D_ok = new FiniteDifferencesDifferent
iator(5, 0.1, 0.0, 1.0);
    FiniteDifferencesDifferentiator D_nb = new FiniteDifferencesDifferent
iator(5, 0.1);
    System.out.println("[PASS] VT05 constructors created successfully.");
    // VT06: differentiate(null) -> NullPointerException
    expectThrow("VT06 differentiate(null)", () -> D_ok.differentiate((Uni
variateFunction) null),
        NullPointerException.class);
    // VT07: g = differentiate(F_CONST5), value(x) inside [a,b]
    headline("VT07 value() on constant");
    UnivariateDifferentiableFunction g07 = diff(D_ok, F_CONST5);
    double y07 = g07.value(0.5);
    checkEquals("g(0.5)", 5.0, y07, 1e-15);
    // VT08: x outside [a,b] -> OutOfRangeException
    headline("VT08 out-of-range (value)");
    expectThrow("x<a", () -> g07.value(-1e-
3), OutOfRangeException.class);
    expectThrow("x>b", () -> g07.value(1.0 + 1e-
3), OutOfRangeException.class);
    // VT09: x=±Inf -> NotFiniteNumberException
    headline("VT09 non-finite x (value)");
    expectThrow("+Inf", () -> g07.value(Double.POSITIVE_INFINITY), NotFin
iteNumberException.class);
    expectThrow("-
Inf", () -> g07.value(Double.NEGATIVE_INFINITY), NotFiniteNumberExcepti
on.class);
    // VT10: x=NaN -> y=NaN
    headline("VT10 NaN x (value)");
    double y10 = g07.value(Double.NaN);
    checkEquals("g(NaN)", Double.NaN, y10, 0.0);
    // VT11: DS path - t=null -> NullPointerException
    headline("VT11 DS null");

```

```

    expectThrow("value((DS)null)", () -> g07.value((DerivativeStructure)
null), NullArgumentException.class);
    // VT12: DS path - t.value outside [a,b]
    headline("VT12 DS out-of-range");
    DerivativeStructure t12 = new DerivativeStructure(1, 1, 0, -1e-3);
    expectThrow("t.value<a", () -> g07.value(t12), OutOfRangeException.cl
ass);
    // VT13: DS path - t.value is ±Inf
    headline("VT13 DS non-finite abscissa");
    DerivativeStructure t13p = new DerivativeStructure(1, 1, 0, Double.PO
SITIVE_INFINITY);
    DerivativeStructure t13n = new DerivativeStructure(1, 1, 0, Double.NE
GATIVE_INFINITY);
    expectThrow("t=+Inf", () -> g07.value(t13p), NotFiniteNumberException
.class);
    expectThrow("t=-
Inf", () -> g07.value(t13n), NotFiniteNumberException.class);
    //VT14: DS path - t.value is NaN -> NaN DS
    headline("VT14 DS NaN abscissa -> NaN DS");
    DerivativeStructure t14 = new DerivativeStructure(1, 2, 0, Double.NaN
);
    DerivativeStructure Y14 = g07.value(t14);
    checkEquals("Y.value", Double.NaN, Y14.getValue(), 0.0);
    // VT15: DS path - order > nPts-1 -> NumberIsTooLargeException
    headline("VT15 DS order > max");
    DerivativeStructure t15 = new DerivativeStructure(1, 5, 0, 0.5); // D
_ok: nPts=5 => max order 4
    expectThrow("order>nPts-
1", () -> g07.value(t15), NumberIsTooLargeException.class);
    // VT16:
    headline("VT16 polynomial accuracy");
    FiniteDifferencesDifferentiator D_poly = new FiniteDifferencesDiffere
ntiator(7, 1e-3, -1.0, 1.0);
    UnivariateDifferentiableFunction g16_quad = diff(D_poly, F_QUAD);
    UnivariateDifferentiableFunction g16_cubic = diff(D_poly, F_CUBIC);
    double x1 = 0.123;
    DerivativeStructure ds1 = new DerivativeStructure(1, 1, 0, x1);
    DerivativeStructure r1 = g16_quad.value(ds1);
    checkEquals("d/dx x^2 @0.123", 2.0 * x1, r1.getPartialDerivative(1),
1e-6);
    DerivativeStructure ds2 = new DerivativeStructure(1, 2, 0, x1);
    DerivativeStructure r2 = g16_quad.value(ds2);
    checkEquals("d2/dx2 x^2", 2.0, r2.getPartialDerivative(2), 1e-5);
    DerivativeStructure ds3 = new DerivativeStructure(1, 3, 0, -0.42);

```

```

DerivativeStructure r3 = g16_cubic.value(ds3);
checkEquals("d3/dx3 x^3", 6.0, r3.getPartialDerivative(3), 1e-4);
// VT17:
headline("VT17 stencil shift near lower bound");
FiniteDifferencesDifferentiator D_low = new FiniteDifferencesDifferentiator(5, 0.1, 0.0, 1.0);
UnivariateDifferentiableFunction g17 = diff(D_low, F_LIN);
DerivativeStructure ds17 = new DerivativeStructure(1, 1, 0, 0.03);
DerivativeStructure r17 = g17.value(ds17);
checkEquals("d/dx (2x-
3) @ near a", 2.0, r17.getPartialDerivative(1), 1e-6);
// VT18:
headline("VT18 stencil shift near upper bound");
DerivativeStructure ds18 = new DerivativeStructure(1, 1, 0, 0.985);
DerivativeStructure r18 = g17.value(ds18);
checkEquals("d/dx (2x-
3) @ near b", 2.0, r18.getPartialDerivative(1), 1e-6);
// VT19:
headline("VT19 sampled NaN propagation");
UnivariateDifferentiableFunction g19 = diff(D_ok, F_NAN);
checkEquals("value(double) NaN", Double.NaN, g19.value(0.5), 0.0);
DerivativeStructure ds19 = new DerivativeStructure(1, 2, 0, 0.5);
DerivativeStructure r19 = g19.value(ds19);
checkEquals("value(DS) NaN", Double.NaN, r19.getValue(), 0.0);
// VT20:
headline("VT20 sin sanity (first derivative ≈ cos)");
FiniteDifferencesDifferentiator D_sin = new FiniteDifferencesDifferentiator(7, 1e-3, 0.0, 1.0);
UnivariateDifferentiableFunction g20 = diff(D_sin, F_SIN);
double xs = 0.4;
DerivativeStructure ds20 = new DerivativeStructure(1, 1, 0, xs);
DerivativeStructure r20 = g20.value(ds20);
checkEquals("d/dx sin(x) @0.4 ≈ cos(0.4)", Math.cos(xs), r20.getPartialDerivative(1), 5e-6);
System.out.println("\n[Done] FiniteDifferencesDifferentiator VT01-VT20 executed.");
}
}
//FT
class FDD_FT {
// ----- helpers -----
private static String fmt(double v) {
if (Double.isNaN(v))
return "NaN";

```

```

if (v == Double.POSITIVE_INFINITY)
    return "+Inf";
if (v == Double.NEGATIVE_INFINITY)
    return "-Inf";
if (v == 0.0) {
    long bits = Double.doubleToRawLongBits(v);
    return (bits >>> 63) == 1 ? "-0.0" : "0.0";
}
return Double.toString(v);
}
private static void sep(String name) {
    System.out.println("\n=====
=====");
    System.out.println(name);
    System.out.println("-----
-----");
}
private static UnivariateDifferentiableFunction diff(FiniteDifferences
Differentiator D, UnivariateFunction f) {
    return D.differentiate((org.apache.commons.math4.legacy.analysis.Univ
ariateFunction) f);
}
private static void dumpDS(UnivariateDifferentiableFunction g, double
x, int order) {
    try {
        DerivativeStructure t = new DerivativeStructure(1, order, 0, x);
        DerivativeStructure y = g.value(t);
        System.out.printf("DS@x=%s value=%s", fmt(x), fmt(y.getValue()));
        for (int k = 1; k <= order; k++) {
            System.out.printf(" d%d=%s", k, fmt(y.getPartialDerivative(k)));
        }
        System.out.println();
    } catch (Throwable t) {
        System.out.println("DS threw: " + t.getClass().getName() + " - " + t
.getMessage());
    }
}
private static void runFT(String tag, int nPts, double h, double a, do
uble b, UnivariateFunction f, double xValue,
    int dsOrder) {
    sep(tag + String.format(" [nPts=%d, h=%s, a=%s, b=%s]", nPts, fmt(h),
fmt(a), fmt(b)));
    try {

```

```

    FiniteDifferencesDifferentiator D = new FiniteDifferencesDifferentia
tor(nPts, h, a, b);
    UnivariateDifferentiableFunction g = diff(D, f);
    try {
        double y = g.value(xValue);
        System.out.printf("value(x=%s) = %s\n", fmt(xValue), fmt(y));
    } catch (Throwable t) {
        System.out.println("value(double) threw: " + t.getClass().getName()
+ " - " + t.getMessage());
    }
    dumpDS(g, xValue, Math.max(1, Math.min(3, dsOrder)));
} catch (Throwable t) {
    System.out.println("ctor/differentiate threw: " + t.getClass().getNa
me() + " - " + t.getMessage());
}
}

private static void runFT_NoBounds(String tag, int nPts, double h, Uni
variateFunction f, double xValue,
    int dsOrder) {
    sep(tag + String.format(" [nPts=%d, h=%s, no-
bounds ctor]", nPts, fmt(h)));
    try {
        FiniteDifferencesDifferentiator D = new FiniteDifferencesDifferentia
tor(nPts, h);
        UnivariateDifferentiableFunction g = diff(D, f);
        try {
            double y = g.value(xValue);
            System.out.printf("value(x=%s) = %s\n", fmt(xValue), fmt(y));
        } catch (Throwable t) {
            System.out.println("value(double) threw: " + t.getClass().getName()
+ " - " + t.getMessage());
        }
        dumpDS(g, xValue, Math.max(1, Math.min(3, dsOrder)));
    } catch (Throwable t) {
        System.out.println("ctor/differentiate threw: " + t.getClass().getNa
me() + " - " + t.getMessage());
    }
}

private static final UnivariateFunction F_POLY5 = x -> {
    double[] c = { -1808425884.0, 1677918208.0, -3.14159, 2.718281828, -
0.57721, 0.125 };
    double y = 0;
    for (int i = c.length - 1; i >= 0; i--)
        y = y * x + c[i];

```

```

    return y;
};
private static final UnivariateFunction F_ALT = x -> {
    double[] c = { 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1 };
    double y = 0;
    for (int i = c.length - 1; i >= 0; i--)
        y = y * x + c[i];
    return y;
};
private static final UnivariateFunction F_SIN_HF = x -> Math.sin(1e3 *
x); // 高频
private static final UnivariateFunction F_SIN_LF = x -> Math.sin(1e-
6 * x); // 低频 (近线性)
private static final UnivariateFunction F_EXP = Math::exp; // 指数 (可
能很大)
private static final UnivariateFunction F_TANH = Math::tanh; // 饱和
private static final UnivariateFunction F_ATAN = Math::atan; // 平滑S
形
private static final UnivariateFunction F_RAT = x -> x == 0 ? 0 : (x /
(1 + x * x)); // 有理函数
private static final UnivariateFunction F_LARGE = x -> 1e308 * x * x;
// 很大值 (注意溢出风险)
private static final UnivariateFunction F_SMALL = x -> 1e-
308 * x * x; // 很小值 (可能下溢到0)
private static final UnivariateFunction F_PW_NAN = x -> (x > 0.2 && x
< 0.25) ? Double.NaN : Math.sin(10 * x);
private static final UnivariateFunction F_COS = Math::cos;
private static final UnivariateFunction F_CONST = x -> -123456789.25;
private static final UnivariateFunction F_LIN = x -> 987654321.0 * x -
24681012.0;
private static final UnivariateFunction F_QUAD = x -> 1e-
4 * x * x - 3e2 * x + 7.0;
private static final UnivariateFunction F_CUBIC = x -> -
2 * x * x * x + 5 * x * x - 7 * x + 11;
private static final UnivariateFunction F_GAUSS = x -> Math.exp(-
x * x / 0.02); // 高斯核 ( $\sigma=0.1$ )
private static final UnivariateFunction F_STEP = x -> (x >= 0) ? 1.0 :
-1.0; // 非光滑 (有限差分可用)
private static final UnivariateFunction F_MIX = x -> Math.sin(x) + 0.1
* Math.sin(50 * x)
+ 0.01 * Math.sin(1000 * x);
public void Ftest() {
    runFT("FT01 poly5 mid", 7, 1e-3, -1.0, 1.0, F_POLY5, 0.123456789, 3);
}

```

```

    runFT_NoBounds("FT02 alt poly no-bounds", 9, 5e-4, F_ALT, -
0.987654321, 3);
    runFT("FT03 sin(1000x)", 9, 1e-4, -0.5, 0.5, F_SIN_HF, 0.049, 3);
    runFT("FT04 sin(1e-6 x)", 7, 1e-2, -1.0, 1.0, F_SIN_LF, -0.333, 2);
    runFT("FT05 exp near lower bound", 5, 0.05, 0.0, 1.0, F_EXP, 0.03, 2)
;
    runFT("FT06 tanh near upper bound", 5, 0.05, 0.0, 1.0, F_TANH, 0.985,
2);
    runFT_NoBounds("FT07 atan large |x|", 7, 1e-3, F_ATAN, 12345.678, 1);
    runFT("FT08 rational around 0", 7, 1e-3, -1.0, 1.0, F_RAT, 1e-6, 3);
    runFT("FT09 huge quad", 7, 1e-3, -1.0, 1.0, F_LARGE, 0.9, 1);
    runFT("FT10 tiny quad", 7, 1e-3, -1.0, 1.0, F_SMALL, 0.9, 1);
    runFT("FT11 piecewise-NaN", 7, 1e-2, 0.0, 1.0, F_PW_NAN, 0.23, 2);
    runFT("FT12 cos mid", 7, 2e-3, -2.0, 2.0, F_COS, -1.2, 3);
    runFT("FT13 constant -1.234e8", 5, 1e-1, -
10.0, 10.0, F_CONST, 3.14, 3);
    runFT("FT14 large-slope linear", 5, 5e-2, -10.0, 10.0, F_LIN, -
7.777, 1);
    runFT("FT15 skewed quad", 9, 1e-3, -5.0, 5.0, F_QUAD, 3.21, 2);
    runFT("FT16 cubic mid", 9, 1e-2, -2.0, 2.0, F_CUBIC, 0.0, 3);
    runFT("FT17 gaussian narrow", 9, 5e-4, -1.0, 1.0, F_GAUSS, 0.05, 3);
    runFT("FT18 sign step", 7, 1e-2, -1.0, 1.0, F_STEP, 1e-6, 1);
    runFT("FT19 mixed sin", 9, 1e-3, -1.0, 1.0, F_MIX, -0.37, 3);
    runFT("FT20 alt poly near upper", 9, 1e-2, 0.0, 1.0, F_ALT, 0.91, 3);
    System.out.println("\n[Done] FiniteDifferencesDifferentiator FT01-
FT20 executed.");
}
}
//Z3
class FDD_Z3Exec {
    /* ----- helpers ----- */
    private static void headline(String name) {
        System.out.println("\n==== " + name + " ===");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expe
cted) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf(Locale.ROOT, "[FAIL] %s should throw %s\n", name,
expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.ge
tMessage());

```

```

        System.out.printf(Locale.ROOT, "[%s] %s threw expected%n", t.getClass().equals(expected) ? "PASS" : "FAIL",
            name);
    }
}

private static String fmt(double v) {
    if (Double.isNaN(v))
        return "NaN";
    if (v == Double.POSITIVE_INFINITY)
        return "+Inf";
    if (v == Double.NEGATIVE_INFINITY)
        return "-Inf";
    if (v == 0.0)
        return (Double.doubleToRawLongBits(v) >>> 63) == 1 ? "-0.0" : "0.0";
    return String.format(Locale.ROOT, "%.17g", v);
}

private static void checkEquals(String name, double expected, double actual, double tolRel) {
    boolean ok;
    if (Double.isNaN(expected) || Double.isNaN(actual)) {
        ok = (Double.isNaN(expected) && Double.isNaN(actual));
    } else if (Double.isInfinite(expected) || Double.isInfinite(actual)) {
        ok = (Double.doubleToRawLongBits(expected) == Double.doubleToRawLongBits(actual));
    } else {
        double tol = Math.max(1.0, Math.max(Math.abs(expected), Math.abs(actual))) * tolRel;
        ok = (expected == actual) || (Math.abs(expected - actual) <= tol);
    }
    System.out.printf(Locale.ROOT, "[%s] %s expected=%s actual=%s tolRel=%.1e%n", ok ? "PASS" : "FAIL", name,
        fmt(expected), fmt(actual), tolRel);
}

private static void checkInt(String name, int exp, int act) {
    System.out.printf(Locale.ROOT, "[%s] %s expected=%d actual=%d%n", (exp == act) ? "PASS" : "FAIL", name, exp,
        act);
}

private static UnivariateDifferentiableFunction diff(FiniteDifferences Differentiator D, UnivariateFunction f) {
    return D.differentiate((org.apache.commons.math4.legacy.analysis.UnivariateFunction) f);
}

```

```

/* ----- sample functions ----- */
private static final UnivariateFunction F_ZERO = x -> 0.0;
private static final UnivariateFunction F_CONST5 = x -> 5.0;
private static final UnivariateFunction F_LIN = x -> 2.0 * x - 3.0; //
f' = 2
private static final UnivariateFunction F_CUBIC = x -> -
2 * x * x * x + 5 * x * x - 7 * x + 11;
private static final UnivariateFunction F_PW_NAN = x -> (x > 2.8 && x
< 3.2) ? Double.NaN : Math.sin(x);
public void Z3test() {
    // Z301) Construct: nPts < 2 → NumberIsTooSmall
    expectThrow("Z301 nPts<2", () -> new FiniteDifferencesDifferentiator(
1, 0.1, 0.0, 1.0),
        NumberIsTooSmallException.class);
    // Z302) Construct: h <= 0 → NotStrictlyPositive
    expectThrow("Z302 h<=0", () -> new FiniteDifferencesDifferentiator(4,
0.0, 0.0, 1.0),
        NotStrictlyPositiveException.class);
    // Z303) Construct: a >= b → NumberIsTooLarge
    expectThrow("Z303 a>=b", () -> new FiniteDifferencesDifferentiator(4,
0.1, 1.0, 1.0),
        NumberIsTooLargeException.class);
    // Z304) Construct: span > b-a → NumberIsTooLarge
    // nPts=20, h=1 => span=19; b-a=10 -> violation
    expectThrow("Z304 span>(b-
a)", () -> new FiniteDifferencesDifferentiator(20, 1.0, 0.0, 10.0),
        NumberIsTooLargeException.class);
    // Z305) Construct: OK → ex=None & span=(nPts-1)*h
    headline("Z305 construct OK");
    FiniteDifferencesDifferentiator D305 = new FiniteDifferencesDifferent
iator(6, 1.0, 0.0, 10.0);
    UnivariateDifferentiableFunction g305 = diff(D305, F_LIN);
    DerivativeStructure t305 = new DerivativeStructure(1, 1, 0, 5.0);
    DerivativeStructure y305 = g305.value(t305);
    checkEquals("Z305 d/dx (2x-
3) @5", 2.0, y305.getPartialDerivative(1), 1e-8);
    // Z306) Differentiate: f=null → NullArgument
    headline("Z306 differentiate(null)");
    expectThrow("Z306 differentiate(null)", () -> D305.differentiate((Uni
variateFunction) null),
        NullArgumentException.class);
    // Z307) Differentiate: OK (wrap f) → ex=None
    headline("Z307 differentiate OK");
    UnivariateDifferentiableFunction g307 = diff(D305, F_CONST5);

```

```

    checkEquals("Z307 g(1.0)", 5.0, g307.value(1.0), 0.0);
    FiniteDifferencesDifferentiator Dn = new FiniteDifferencesDifferentiator(100, 1.0 / 99.0, 0.0, 10.0);
    UnivariateDifferentiableFunction g_lin_narrow = diff(Dn, F_LIN);
    // Z308) Eval0: x is NaN → ex=None & y is NaN
    headline("Z308 value(NaN)");
    double z308 = g_lin_narrow.value(Double.NaN);
    checkEquals("Z308 y", Double.NaN, z308, 0.0);
    // Z309) Eval0: x=±∞ → NotFiniteNumber
    headline("Z309 value(+Inf)");
    expectThrow("Z309 +Inf", () -> g_lin_narrow.value(Double.POSITIVE_INFINITY), NotFiniteNumberException.class);
    // Z310) Eval0: x out of [a,b] → OutOfRange
    headline("Z310 value out-of-range");
    expectThrow("Z310 x<a", () -> g_lin_narrow.value(-1.0), OutOfRangeException.class);
    // Z311) Eval0: in-range & finite samples → ex=None (finite y)
    headline("Z311 value finite");
    double z311 = g_lin_narrow.value(5.0);
    checkEquals("Z311 y", 2.0 * 5.0 - 3.0, z311, 1e-12);
    // Z312) Eval0: in-range & sampled NaN → ex=None & y is NaN
    headline("Z312 sampled NaN");
    UnivariateDifferentiableFunction g312 = diff(Dn, F_PW_NAN); // 在 [2.8, 3.2] 采样为 NaN
    double z312 = g312.value(3.0); // span=1, 会命中 NaN
    checkEquals("Z312 y", Double.NaN, z312, 0.0);
    // Z313) EvalDS: t=null → NullArgument
    headline("Z313 value((DS)null)");
    expectThrow("Z313 DS=null", () -> g_lin_narrow.value((DerivativeStructure) null), NullArgumentException.class);
    // Z314) EvalDS: t.value is NaN → ex=None & T NaN
    headline("Z314 DS with NaN value");
    DerivativeStructure t314 = new DerivativeStructure(1, 2, 0, Double.NaN);
    DerivativeStructure y314 = g_lin_narrow.value(t314);
    checkEquals("Z314 T.value", Double.NaN, y314.getValue(), 0.0);
    // Z315) EvalDS: t.value=±∞ → NotFiniteNumber
    headline("Z315 DS with +Inf value");
    DerivativeStructure t315 = new DerivativeStructure(1, 1, 0, Double.POSITIVE_INFINITY);
    expectThrow("Z315 t=+Inf", () -> g_lin_narrow.value(t315), NotFiniteNumberException.class);
    // Z316) EvalDS: t.value out of [a,b] → OutOfRange
    headline("Z316 DS out-of-range");

```

```

    DerivativeStructure t316 = new DerivativeStructure(1, 1, 0, 11.0);
    expectThrow("Z316 t> b", () -> g_lin_narrow.value(t316), OutOfRangeException.class);
    // Z317) EvalDS: order > nPts-1 → NumberIsTooLarge
    headline("Z317 DS order > max");
    FiniteDifferencesDifferentiator D317 = new FiniteDifferencesDifferentiator(4, 1.0, 0.0, 10.0); // max order = 3
    UnivariateDifferentiableFunction g317 = diff(D317, F_ZERO);
    DerivativeStructure t317 = new DerivativeStructure(1, 5, 0, 5.0); // order=5
    expectThrow("Z317 order>nPts-1", () -> g317.value(t317), NumberIsTooLargeException.class);
    // Z318) EvalDS: in-range & finite samples → ex=None
    headline("Z318 DS finite (orders up to 3)");
    FiniteDifferencesDifferentiator D318 = new FiniteDifferencesDifferentiator(6, 1.0, 0.0, 10.0);
    UnivariateDifferentiableFunction g318 = diff(D318, F_CUBIC);
    DerivativeStructure t318 = new DerivativeStructure(1, 3, 0, 4.0);
    DerivativeStructure y318 = g318.value(t318);
    double d1_expected = -6 * 4.0 * 4.0 + 10 * 4.0 - 7; // f'(x) = -6x^2 + 10x - 7
    checkEquals("Z318 d1", d1_expected, y318.getPartialDerivative(1), 1e-4);
    // Z319) StencilShift: left clamp (x < a + span/2) → center=a+span/2
    headline("Z319 left clamp (near lower bound)");
    FiniteDifferencesDifferentiator D319 = new FiniteDifferencesDifferentiator(7, 0.2, 0.0, 10.0);
    UnivariateDifferentiableFunction g319 = diff(D319, F_LIN);
    DerivativeStructure t319 = new DerivativeStructure(1, 1, 0, 0.1);
    DerivativeStructure y319 = g319.value(t319);
    checkEquals("Z319 d/dx ≈ 2", 2.0, y319.getPartialDerivative(1), 1e-6);
    // Z320) StencilShift: right clamp (x > b - span/2) → center=b-span/2
    headline("Z320 right clamp (near upper bound)");
    DerivativeStructure t320 = new DerivativeStructure(1, 1, 0, 9.9);
    DerivativeStructure y320 = g319.value(t320);
    checkEquals("Z320 d/dx ≈ 2", 2.0, y320.getPartialDerivative(1), 1e-6);
    System.out.println("\n[Done] FDD Z3 executable cases Z301-Z320.");
}
}
class FDD_VTestRunner {
    public static void main(String[] args) {
        // VT

```

```

new FDD_VT().Vtest();
// FT
// new FDD_FT().Ftest();
// Z3
// new FDD_Z3Exec().Z3test();
}
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
FiniteDifferencesDifferentiator.java	27.5 %	38	100	138
FDD_VT	58.8 %	20	14	34
FiniteDifferencesDifferentiator	100.0 %	14	0	14
FiniteDifferencesDifferentiator(int, double)	100.0 %	6	0	6
evaluate(DerivativeStructure, double)	100.0 %	8	0	8
FiniteDifferencesDifferentiator(int, double)		0	0	0
differentiate(UnivariateFunction)		0	0	0
FDD_VTestRunner		0	0	0

3. V-Method testing instruction coverage: 92.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
FiniteDifferencesDifferentiator.java	33.9 %	977	1,901	2,878
FDD_VTestRunner	62.5 %	5	3	8
FDD_VT	87.1 %	660	98	758
FiniteDifferencesDifferentiator	92.7 %	230	18	248
FiniteDifferencesDifferentiator(int, double)	100.0 %	7	0	7
FiniteDifferencesDifferentiator(int, double)	100.0 %	84	0	84
differentiate(UnivariateFunction)	100.0 %	6	0	6
evaluate(DerivativeStructure, double)	100.0 %	133	0	133

4. Fuzz testing branch coverage: 78.6%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
FiniteDifferencesDifferentiator.java	24.6 %	34	104	138
FiniteDifferencesDifferentiator	78.6 %	11	3	14
FiniteDifferencesDifferentiator(int, double)	50.0 %	3	3	6
evaluate(DerivativeStructure, double)	100.0 %	8	0	8
FiniteDifferencesDifferentiator(int, double)		0	0	0
differentiate(UnivariateFunction)		0	0	0
FDD_FT	83.3 %	20	4	24
FDD_VTestRunner		0	0	0

5. Fuzz testing instruction coverage: 81%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
FiniteDifferencesDifferentiator.java	34.3 %	986	1,892	2,878
FDD_VTestRunner	62.5 %	5	3	8
FiniteDifferencesDifferentiator	81.0 %	201	47	248
FiniteDifferencesDifferentiator(int, double)	65.5 %	55	29	84
FiniteDifferencesDifferentiator(int, double)	100.0 %	7	0	7
differentiate(UnivariateFunction)	100.0 %	6	0	6
evaluate(DerivativeStructure, double)	100.0 %	133	0	133
FDD_FT	88.4 %	710	93	803

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ FiniteDifferencesDifferentiator.java	27.5 %	38	100	138
> FDD_Z3Exec	52.6 %	20	18	38
▼ FiniteDifferencesDifferentiator	100.0 %	14	0	14
FiniteDifferencesDifferentiator(int, d	100.0 %	6	0	6
evaluate(DerivativeStructure, double	100.0 %	8	0	8
differentiate(UnivariateFunction)		0	0	0
> FDD_VTestRunner		0	0	0

7. Z3 instruction coverage: 89.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ FiniteDifferencesDifferentiator.java	31.5 %	908	1,970	2,878
> FDD_VTestRunner	62.5 %	5	3	8
> FDD_Z3Exec	85.1 %	598	105	703
▼ FiniteDifferencesDifferentiator	89.9 %	223	25	248
FiniteDifferencesDifferentiator(int, d	100.0 %	84	0	84
differentiate(UnivariateFunction)	100.0 %	6	0	6
evaluate(DerivativeStructure, double	100.0 %	133	0	133